Journal of Network and Computer Applications 238 (2025) 104169

Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca

Research paper

A MLOps architecture for near real-time distributed Stream Learning operation deployment

Miguel G. Rodrigues, Eduardo K. Viegas *, Altair O. Santin, Fabricio Enembreck

Pontificia Universidade Catolica do Parana (PUCPR), Graduate Program in Computer Science (PPGIa), Brazil

ARTICLE INFO

Keywords:
MLOps
Stream Learning
Kubernetes
Microservices

ABSTRACT

Traditional architectures for implementing Machine Learning Operations (MLOps) usually struggle to cope with the demands of Stream Learning (SL) environments, where deployed models must be incrementally updated at scale and in near real-time to handle a constantly evolving data stream. This paper proposes a new distributed architecture adapted for deploying and updating SL models under the MLOps framework, implemented twofold. First, we structure the core components as microservices deployed on a container orchestration environment, ensuring low computational overhead and high scalability. Second, we propose a periodic model versioning strategy that facilitates seamless updates of SL models without degrading system accuracy. By leveraging the inherent characteristics of SL algorithms, we trigger the model versioning task only when their decision boundaries undergo significant adjustments. This allows our architecture to support scalable inference while handling incremental SL updates, enabling high throughput and model accuracy in production settings. Experiments conducted on a proposal's prototype implemented as a distributed microservice architecture on Kubernetes attested to our scheme's feasibility. Our architecture can scale inference throughput as needed, delivering updated SL models in less than 2.5 s, supporting up to 8 inference endpoints while maintaining accuracy similar to traditional single-endpoint setups.

1. Introduction

The Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computing systems to learn from data without explicit programming (Neto et al., 2024). The applications of ML range from process automation and chatbots to supply chain optimization, targeted marketing, cybersecurity, and autonomous vehicles. It involves creating models that identify and generalize patterns in data to predict new, unseen inputs (Pruneski et al., 2022). Supervised learning, the most common ML application, builds models on labeled datasets to solve classification or regression problems, where the goal is to categorize data or predict continuous values (Chaoji et al., 2016). Building and deploying ML models typically involves several steps. These include data collection, preprocessing, model training, validation, deployment, performance monitoring, and updates to maintain accuracy.

Production management of a ML model, referred to as MLOps, is an iterative task involving multiple disciplines that require adequately managing the processes involved (Ashmore et al., 2021). It is a collaborative approach that streamlines the development task by automating workflows, managing model versions, standardizing code, and ensuring scalability, ultimately improving security and compliance in production environments (Burgueño-Romero et al., 2025). In this context, a critical MLOps phase is model deployment, called inference, where trained models predict new data, often remotely accessed by multiple users at scale, as in credit card fraud detection and recommendation systems (Shinde and Shah, 2018). To support such requirements, distributed architectures are built using tools such as MLflow (MLflow Project, 2018) for tracking and versioning, AWS SageMaker (Amazon Web Services, 2017), and Google Cloud AI Platform (Google, 2018) for providing ML frameworks, and technologies such as microservices and containerization for adaptive system design (Bentaleb et al., 2021).

A typical MLOps framework must implement four key components, namely *inference*, *model update*, *versioning*, and *monitoring* modules. The *inference* module loads ML models, handles queries, performs predictions, and returns results. The *model update* module manages retraining. It uses labeled events at scheduled intervals to ensure the deployed model remains current and reliable. The *versioning* module stores model versions generated during training and retraining, making them available to inference modules when needed. Finally, the *monitoring* module

* Corresponding author.
E-mail addresses: miguel.rodrigues@ppgia.pucpr.br (M.G. Rodrigues), eduardo.viegas@ppgia.pucpr.br (E.K. Viegas), santin@ppgia.pucpr.br (A.O. Santin), fabricio.enembreck@ppgia.pucpr.br (F. Enembreck).

https://doi.org/10.1016/j.jnca.2025.104169
Received 23 November 2024; Received in revised form 14 February 2025; Accepted 10 March 2025
Available online 17 March 2025

evaluates the performance of the deployed model in production, comparing it to new versions and allowing it to be replaced if an improved version is identified (Burgueño-Romero et al., 2025).

Current MLOps architectures generally meet the needs of traditional ML scenarios where data patterns evolve incrementally. However, traditional MLOps practices may fall short in SL environments, as they are not designed to handle incremental updates or to deliver updated models in very short time frames. SL refers to ML algorithms that continuously update models with incoming data streams without having to perform multiple passes over the data (Gomes et al., 2017). These data streams often come from various sources and are characterized by unbounded size, high and variable arrival rates, changing data patterns, and memory constraints during processing (Agrahari and Singh, 2022). These characteristics present challenges that may not be adequately addressed by traditional MLOps. In a SL-enabled architecture, the inference process must handle many requests and often relies on parallelism techniques to manage extensive data flows (Upadhyaya, 2013). While similar to traditional MLOps systems, a SL architecture uniquely requires constantly replacing models in memory with the latest versions, posing significant challenges, particularly when the number of endpoints or the model replacement rate is high (Kiaee and Arianyan, 2025).

Frequent model versioning in SL is challenging because loading new model versions to the endpoints and serializing those versions must be done quickly (Garg et al., 2021). The versioning engine must also be able to efficiently manage the large number of model versions generated by these frequent updates. In contrast to traditional MLOps, where updates are typically performed in batches or mini-batches, model updates in SL environments need to be incremental and continuous. Ensuring the model remains available to inference endpoints is not easily achieved when their parameters are updated with each new event. This is because endpoints require significant time and computational resources to deserialize the updated model before inference can proceed, which can disrupt the inference service (Yadwadkar et al., 2019).

Surprisingly, there is a lack of research on developing new SL-oriented architectures that address the challenges of near real-time model updates within the MLOps framework (Burgueño-Romero et al., 2025; Barry et al., 2023). Distributed MLOps architectures for SL face the challenge of performing model updates in near real-time, ensuring that new events are efficiently incorporated into the model without disrupting inference performance. This requires continuous, incremental model updates with minimal latency while maintaining the overall system's responsiveness and scalability across distributed infrastructure (Suárez-Cetrulo et al., 2023). Current literature generally focuses on scaling either model updates or inference, often overlooking their integration within an MLOps framework (Bifet et al., 2015). This becomes particularly challenging when model versioning is introduced, as frequent model updates typically halt inference to perform model deserialization. As a result, the impact of continuous model updates on the system's predictive performance over time is often only partially addressed or overlooked in existing studies (Komisarek et al., 2020; Paleyes et al., 2022). Most approaches rely on traditional MLOps update procedures that perform periodic batch-oriented updates, which can ultimately result in performance degradation (Markov et al., 2022). Other approaches use concept drift detection strategies, triggering updates only after data pattern changes or model performance declines are detected (Lu et al., 2018). Thus, these methods often render the updated model obsolete upon deployment and typically require continuous expert supervision. As a result, MLOps adoption for SL tasks in production environments has been hindered by the scarcity of literature and the absence of tools and procedures that support continuous inference and model updates on data streams.

Contribution. Our work paves the way for a scalable architecture for deploying SL models in production environments under the MLOps framework. The goal of the proposed scheme is to incrementally update the SL models and make them available with a configurable frequency while simultaneously handling the inference requests of the users at scale in a decentralized manner. The implementation of our proposed architecture is twofold. First, its core components are implemented as microservices in a container orchestration environment. This allows for low computational cost, high portability, and adaptability. Second, a dynamic model versioning approach is introduced. This enables new SL models to be made available to inference endpoints without affecting the accuracy performance of the system. Our approach leverages the inherent characteristics of SL algorithms by triggering the model versioning task only when significant adjustments occur in their decision boundaries. As a result, our scheme enables horizontal scaling of the inference module and supports SL incremental updates and versioning through incoming data flows. Each component of the architecture is designed to provide a high inference throughput without compromising the accuracy of the resulting model.

In summary, the main contributions of the paper are as follows:

- A new distributed architecture that enables SL-oriented MLOps deployment at scale. Our proposed scheme offers a configurable deployment frequency for new models to inference endpoints without affecting the overall accuracy and inference throughput performance;
- A proposal prototype implemented as microservices running on a container orchestration environment. Our architecture can scale the inference throughput as needed, delivering updated SL models in less than 2.5 s, supporting up to 8 inference endpoints, while maintaining an accuracy similar to that of traditional single endpoint setups

Roadmap. The remainder of this paper is organized as follows. Section 2 describes the fundamentals behind SL and MLOps. Section 3 overviews the related works on MLOps implementation. Section 4 describes our proposed solution, Section 5 introduces our prototype, and Section 6 evaluates its performance. Finally, Section 7 concludes our work.

2. Preliminaries

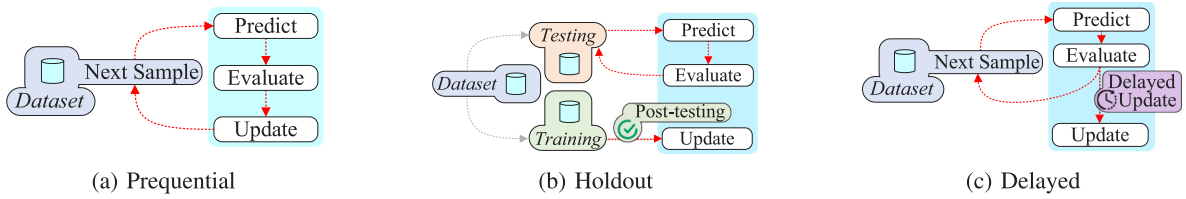
Deployment of SL techniques in production environments has increased substantially in recent years. This section covers the fundamentals of SL, followed by an overview of MLOps operations for SL-oriented systems. Finally, we review the challenges traditional MLOps implementations typically face in SL environments.

2.1. Stream Learning (SL)

In contrast to traditional batch-oriented ML, where models are static and built using often immutable historical data, SL involves data that is constantly changing its patterns in response to changes in the environment (Lu et al., 2018). In a streaming environment, data is generated in a continuous, ordered flow that is potentially infinite and is transmitted at high speed, with examples arriving as a stream of data S.

Let $x_i \in X$ be an event, where $x_i \in \mathbb{R}^D$ denotes a D-dimensional feature vector input for the ith event. The goal of the SL system is to continuously update a classifier function $f(x) : x \rightarrow y$ that outputs the predicted class y for a given input feature vector x. Here, the SL classifier function is usually updated incrementally such that it minimizes the resulting model's error rate. In practice, individual events must make model updates without access to past events.

This is because the data in these scenarios can only be accessed once or retained for a short time, so each step of SL must be performed quickly and resource-efficiently (Barddal et al., 2017). Since SL models



(a) Prequential (b) Holdout (c) Delayed

Fig. 1. Evaluation approaches for stream learning classifiers. *Prequential* evaluation tests each incoming sample before using it for training. *Holdout* evaluation periodically tests the model on a fixed subset of data not used for training. *Delayed* evaluation tests each incoming sample and uses them for updates after a specified delay.

update with each new training instance, they naturally adapt to changing data concepts. In this case, for detecting abrupt concept shifts, a concept drift detector may be necessary for faster adaptation. Unlike traditional batch-oriented schemes, SL models are generally better suited to adapt to these inherent changes in data flows (Gonçalves et al., 2014).

Balancing the retention of prior knowledge while adapting to new concepts during incremental SL model updates is a challenging task (Moya et al., 2023). This challenge arises because the decision boundaries of SL classifiers are continuously updated to accommodate new events. A widely adopted approach in the design of SL-oriented classifiers is incorporating a grace period parameter to tackle this issue. This parameter ensures that significant adjustments to the classifier's decision boundaries occur only after processing sufficient events for incremental training. For instance, the Hoeffding Tree classifier (Banar et al., 2023) splits a leaf node only after it has observed a predefined number of events, as specified by the grace period parameter. As a result, although incremental model updates are conducted, the decision boundaries of SL classifiers are usually only substantially affected according to the previously defined grace period parameter.

Evaluating a SL classifier is inherently challenging due to data streams' continuous and evolving nature (Gama et al., 2012). Researchers typically adopt one of three main approaches for evaluation, namely *prequential*, *holdout*, or *delayed*, as shown in Fig. 1. Prequential evaluation involves testing each incoming sample before using it for training, enabling real-time performance assessment but requiring careful handling to avoid bias in evolving data distributions. On the other hand, holdout evaluation relies on periodically testing the model against a fixed subset of data not used for training, providing a stable reference for comparison but potentially overlooking concept drift in the stream. Lastly, delayed evaluation assesses the model's performance on a batch of data after it has been trained on earlier samples, introducing a controlled delay to account for temporal dependencies while balancing computational and memory constraints. Each approach offers distinct advantages and trade-offs, making the evaluation method important for ensuring accurate and meaningful performance assessments in SL.

Managing the life-cycle of a SL model is more complex than that of a traditional ML model, as online systems need to handle continuous data ingestion while at the same time ensuring near real-time model update and inference, usually in a distributed setting (Gomes et al., 2017). As a result, despite the increased efficiency in various applications, implementing SL in production environments remains a significant challenge for ML professionals (Gomes et al., 2019). These challenges make SL more suitable for academic environments, where the scenarios can be more easily controlled, and the delay in inference is less critical than that observed in commercial environments. This situation often leads companies to rely on near-real-time learning in streaming scenarios to partially mitigate the obstacles associated with SL (Anandan et al., 2015).

2.2. Machine Learning Operations (MLOps)

MLOps is a practice designed to optimize the development, deployment, monitoring, and ongoing management of ML-based systems in production environments. It extends the principles of Development and Operations (DevOps) to address the unique challenges of the ML model life-cycle, ensuring that they are successfully deployed and maintained in production while efficiently updating them as new data arrives (Burgueño-Romero et al., 2025). This approach combines provision, monitoring, and update processes to maintain model performance and proactively address concept drift by integrating practices and tools tailored to the ML life-cycle.

The MLOps operation is typically carried out through the following phases:

- *Provision*. It involves deploying the trained ML model into production, ensuring it has the resources and infrastructure to be queried by multiple users at scale. This is typically achieved by deploying multiple endpoints as web services in a distributed environment that executes the inference phase in parallel to handle user requests;
- Monitoring. It continuously tracks the model's performance and detects concept drift or accuracy degradation issues. It is used to ensure that the deployed model maintains the expected accuracy despite possible changes in the behavior of the deployed environment;
- *Update*. It modifies or retrains the deployed model based on new environment data or changing requirements to maintain system accuracy. On a traditional ML setting process, it is usually triggered periodically based on the conditions identified by the *monitoring* module;
- *Versioning*. It manages the model's different iterations, including keeping track of parameter changes and facilitating possible rollbacks. It aims to allow for possible iterations across the different stages of deployment;

Meeting the requirements of the operational phase at scale is a challenging task that often requires multiple frameworks that can operate in a distributed environment. These include tools such as AirFlow (The Apache Software Foundation, 2014), Kubeflow (The Kubeflow Authors, 2018), and Seldon (Seldon Technologies Limited, 2014), which provide orchestration and automation across the ML life-cycle, while others such as MLflow (MLflow Project, 2018) to focus on specific phases such as experiment tracking and model versioning.

Given the diverse requirements of MLOps, service providers are also implementing platforms such as AWS SageMaker, Azure Machine Learning (Microsoft Corporation, 2018), and Google Cloud AI Platform, which offer integrated solutions to manage the entire MLOps life-cycle. Implementing MLOps is necessary to ensure the proper scaling and maintenance of ML systems in production, reduce errors, save time, and improve governance by ensuring that these systems are effectively meeting the organization's needs.

2.3. When stream learning meets MLOps

Given the increasing use of SL techniques in recent years, several tools have been proposed for deploying designed models in production. For example, Massive Online Analysis (MOA) (The University of Waikato, 2010) provides a framework for the implementation of algorithms and the execution of experiments on evolving data

streams. Similarly, SAMOA (Apache Software Foundation, 2014c) aims to extend MOA's capabilities for mining large data streams through a distributed architecture. It supports classification, clustering, and regression (Pruneski et al., 2022), and is mainly used in academic environments, although it can be interfaced with tools like Apache Storm (Apache Software Foundation, 2014) and Apache Samza (Apache Software Foundation, 2018).

In commercial environments, frameworks such as Scikit-multiflow (Montiel et al., 2018), Creme (Python Software Foundation, 2020), and River (Montiel et al., 2021) are gaining popularity. Scikit-multiflow offers a streaming version of Scikit-learn (Pedregosa et al., 2011), while Creme is known for its simplicity, and River combines both strengths by offering a wide range of SL algorithms. Unfortunately, while they allow for incremental updates, they typically do not cover the entire SL lifecycle, especially when scalability is required, which demands the utilization of additional tools for full production deployment.

Implementing MLOps for SL algorithms presents many challenges that current tools struggle to address. The SL *provision* requires near real-time model access, which makes it difficult to dynamically allocate resources while maintaining low latency, especially when provisioning must occur for each new event. Frequent deserialization of updated models is particularly challenging in near real-time, as the classification task can only be performed after the updated model version has been adequately parsed. Additionally, relying on concept drift detection to trigger model updates introduces further complexity, as the drift detection algorithm must be carefully tuned to the specific dataset and deployment environment, adding a layer of dependency and making the process less adaptable to varying conditions.

The need for continuous model updates triggered for each new event to adapt to evolving data further complicates the deployment task. Each update requires model serialization, versioning, and deserialization within the deployment modules. In addition, managing multiple versions of a model in a streaming context is complex due to the constant flow of data, and ensuring seamless versioning and reproducibility can lead to an unreliable infrastructure as each new event can potentially change the model. In this context, the near real-time demands of SL algorithms are typically not addressed by current MLOps architectures.

3. Related works

Developing new architectures for MLOps operations has been a widely studied topic in the literature over the past years. However, SL-oriented MLOps architectures are still in their infancy. For example, StreamDM (Bifet et al., 2015) enables distributed data processing for SL tasks, but its near real-time effectiveness is limited because it relies on mini-batches, which introduces latency. SOLMA (Jamil et al., 2018), a component of the Apache Flink ecosystem (Apache Software Foundation, 2014a), provides scalability, fault tolerance, and parallel processing support but lacks model versioning capabilities despite enabling near real-time inference. A microservices-based architecture for real-time analytics, where models are trained in batches and updated in mini-batches, is proposed by Xu et al. (2015). This architecture is implemented on Apache Spark (Apache Software Foundation, 2013) and the MLlib library (Apache Software Foundation, 2014b) to support parallel processing and ensure scale performance. However, SL-oriented model updates are neglected. Similarly, Markov's et al. (Markov et al., 2022) streamlines the ML life-cycle but limits model updates to batch processing, a feature that may not be suitable for all SL applications.

Typically, proposed MLOps architectures are targeted at specific use cases, limiting their broad adoption. As an example, a packaging and deployment mechanism for analytic pipelines has been proposed by Minon et al. (2022) for streaming pipelines. It supports distributed deployment through dockers and is decoupled from the training phase to avoid integrating experiment code into operational deployments (Fayos-Jordan et al., 2020). Unfortunately, although it supports streaming model inference, it fails to address near real-time model updates. Similarly, Ramanath et al. (2021) predict ad click rates by combining batch processing with mini-batch learning while adapting to data variations over time using a lambda architecture. Their approach performs online training with mini-batches and requires code adaptations for other use cases. Carcillo et al. (2018) integrated several Big Data tools with ML to provide near real-time credit card fraud detection using random forests and sliding windows for model updates. Its structure cannot be easily customized, and its applicability is limited to specific scenarios. Similarly, Kranjc et al. (2017) presented a web application that supports data mining workflows through a graphical user interface, enabling workflows to be executed in streaming environments using daemons. The near real-time capabilities of their system are limited because training is performed in batch mode.

It is a common practice in the literature to rely on well-known frameworks for stream processing. The Big Data Engine tool developed by Komisarek et al. (2020) integrates Apache Spark, Apache Kafka (Apache Software Foundation, 2011), Elasticsearch (Elastic, 2010), Kibana (Elastic, 2013), and HDFS (Apache Software Foundation, 2006b) for network anomaly detection. It supports near real-time processing and vertical scaling. However, it does not detail model update mechanisms; instead, it focuses on training with data flow. Similarly, Spring XD, presented by Anandan et al. (2015), is module-based and interacts with technologies such as Apache Spark, Apache Kafka, RabbitMQ (Pivotal Software, 2007), Hadoop (Apache Software Foundation, 2006a), Redis (Redis Labs, 2009), and Apache Zookeeper (Apache Software Foundation, 2010). It supports near real-time inference via Spark Stream and includes monitoring tools. However, it performs model updates using Spark's mini-batch mode. Another module-based system, STREAMER, introduced by Garcia-Rodriguez et al. (2020), integrates with Apache Kafka, InfluxDB (InfluxData, 2013), Redis, and Kibana. It provides scalability and fault tolerance while allowing users to focus on algorithms and data preprocessing. Unfortunately, it does not support incremental updates and relies on mini-batches or the processing of entire datasets for model updates. Kafka-ML, presented by Martín et al. (2022), manages ML pipelines with TensorFlow (Google Brain Team, 2015) and PyTorch (PyTorch Foundation, 2016). It provides a web interface for training and inference with Docker containerization (Bentaleb et al., 2021) for portability while processing data in batches rather

Table 1
A summary of related work and the characteristics of their MLOps implementations.

Work	Streaming architecture	Streaming update	Streaming processing	General purpose	Distributed architecture	Model versioning
Bifet et al. (2015)	✓	×	×	✓	✓	×
Jamil et al. (2018)	✓	✓	✓	✓	×	×
Xu et al. (2015)	×	×	×	✓	✓	×
Markov et al. (2022)	×	×	✓	×	×	×
Minon et al. (2022)	×	×	✓	✓	×	×
Ramanath et al. (2021)	×	×	✓	×	×	×
Carcillo et al. (2018)	×	×	×	×	✓	×
Kranjc et al. (2017)	×	×	✓	✓	✓	✓
Komisarek et al. (2020)	✓	×	×	×	✓	×
Anandan et al. (2015)	×	×	×	✓	×	×
Garcia-Rodriguez et al. (2020)	×	×	✓	✓	×	✓
Martín et al. (2022)	×	×	✓	✓	✓	✓
Crankshaw et al. (2017)	×	×	×	✓	✓	×
Baylor et al. (2017)	×	×	✓	✓	✓	✓
Pareek et al. (2019)	×	×	✓	✓	×	×
Ours	✓	✓	✓	✓	✓	✓

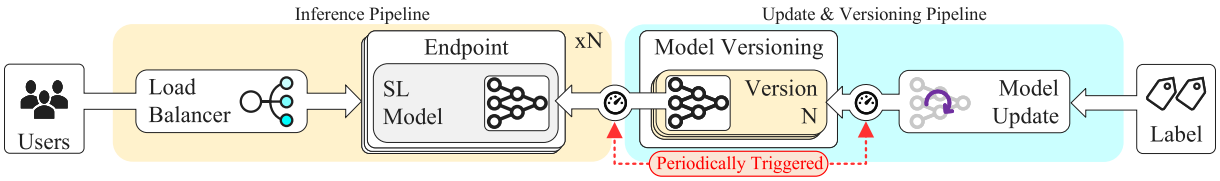
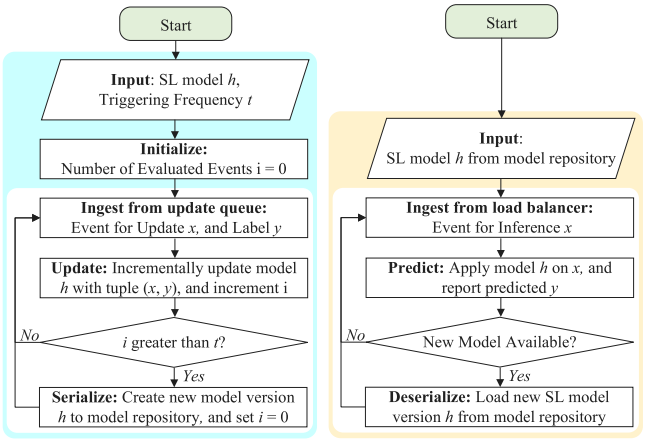


Fig. 2. An overview of the proposed pipeline for implementing the MLOps operational phase in SL-oriented applications. The *Inference Pipeline* consists of multiple endpoints designed to efficiently handle user requests at scale in a continuously dynamic data stream. The *Model Update and Versioning Pipeline* employs a periodic triggering mechanism based on an event counter to generate and deploy new SL model versions.

than incrementally. Here, trained models cannot be updated but saved or deployed for future use.

Crankshaw et al. (2017) presented the Clipper tool, a low-latency prediction system for near real-time online deployments. It supports multiple ML models and deployment strategies through a modular architecture and unified interface. It also provides a dynamic load-balancing ingestion strategy for the even distribution of prediction tasks. However, it does not support incremental updates via a mini-batch approach. Baylor et al. (2017) introduced TensorFlow Extended (TFX), an open-source platform for managing ML pipelines. It integrates analyzing, transforming, validating, and deploying components into a unified system. TFX supports continuous model updates and transfer learning. This reduces training time by transferring parameters from a base to a target network. The feasibility of incremental updates is still being discussed, although continuous updates are supported. Pareek et al. (2019) presents Striim, an enterprise-grade platform for near real-time data ingestion and ML that uses models such as random forests and Gaussian process regression and makes use of sliding windows for model updates. StreamMLOps, proposed by Barry et al. (2023), provides an architecture for online learning. It uses open-source tools such as River and MLFlow. It supports continuous learning and deployment but requires customization for specific applications.



(a) Incremental update pipeline (b) Inference pipeline

Fig. 3. Pipeline for both inference and incremental updates. The *Inference* pipeline continuously ingests events for inference and dynamically deserializes new SL model versions as they become available. The *Incremental Update* pipeline incrementally updates the model and serializes it based on a predefined triggering frequency.

3.1. Discussion

Table 1 reviews the literature on MLOps support for SL-oriented operations. It can be observed that most of the studies focus on updating the model offline. In this approach, labeled data is stored and periodically used to generate models more consistent with the latest information. This approach is usually based on batch processing, where models are retrained regularly to incorporate new data. At the same time, near real-time updates occur at discrete intervals rather than incrementally and often rely on sliding windows to create mini-batches for model building. Despite the challenges associated with SL support, traditional MLOps practices typically involve periodic updates through batch or mini-batch strategies.

Nevertheless, when scaling their designed systems, it is common practice in the literature to deploy additional peers using virtual machines (Bifet et al., 2015; Baylor et al., 2017). However, this approach introduces significant computational overhead due to the need to deploy an entire operating system and the required libraries. In contrast, the use of containers for infrastructure scaling in MLOps is still in its early stages in the literature (Garcia-Rodriguez et al., 2020; Martín et al., 2022). Containers have the potential to significantly reduce service provisioning overhead by sharing the host OS libraries and binaries while utilizing namespaces to ensure isolation (Horchulhack et al., 2024).

Several challenges are associated with deploying SL models in a production environment. These include maintaining system responsiveness at scale while ensuring an efficient inference model under high prediction demand. Traditionally, researchers scale their training procedures in a classifier-dependent manner, which poses a challenge in adapting the architecture to different classifiers. Designing an architecture that enables the training task to be scaled without modifying the supporting architecture remains a significant challenge in the literature.

Furthermore, using traditional MLOps architectures is not feasible due to the need for incremental updates as soon as event labels become available. Current MLOps architectures cannot easily manage the multiple versions of constantly evolving SL models while making them available for inference.

4. A MLOps architecture towards stream learning deployment

In light of this, we propose a new architecture that paves the way for implementing SL-based applications under the MLOps operating principle at scale. Our proposed architecture addresses three key challenges associated with SL-oriented MLOps environments:

- *High Inference Throughput*. To accommodate the evolving demand, the architecture introduces a horizontal scaling system that allows the number of inference endpoints to scale as required. In addition, a decoupled model loading method is implemented, enabling the inference module to retrieve new models directly from the model repository without requiring endpoint restarts, enhancing system availability in near real-time applications;
- *Trigger-based model updates*. In contrast to traditional ML frameworks that rely on batch or mini-batch updates, our proposed architecture integrates a triggering system for incremental SL model updates via a continuous data flow. To achieve this goal, we introduce a model serialization window that is triggered periodically, allowing updated models to be serialized in the model repository and made available for inference within optimal time or event windows;
- *Decoupled Model Versioning*. The frequent model update and versioning can result in hundreds or even thousands of model versions within a short time frame. To address such a challenge,

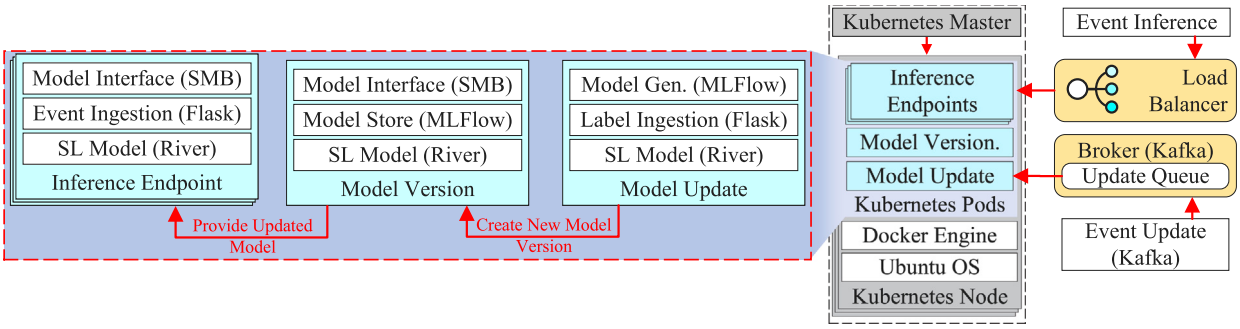


Fig. 4. Prototype overview of our proposed pipeline for implementing the MLOps operation phase on SL-oriented applications.

the architecture decouples the model versioning task to handle multiple model versions while also enabling a faster inference module's loading of specific model versions;

Fig. 2 illustrates our proposal pipeline implementation. It includes the *Inference Pipeline* and the *Model Update and Versioning Pipeline*. The former handles user prediction requests at scale, while the latter manages the incremental update of the SL model using labeled events and appropriate model versioning.

The Inference Pipeline implementation aims to handle multiple user inference requests simultaneously at scale. It implements multiple endpoints executing the latest available SL model version to achieve this goal. Each endpoint is queried through a load balancer. This load balancer efficiently distributes the user load across all available endpoints. As a result, the inference can be scaled by loading multiple model versions on each endpoint, expanding compute capacity as needed.

The Model Update and Versioning Pipeline receives event labels for incremental model updates following a specific process. These labels are used for the incremental SL update while the pipeline initiates model versioning periodically. We designed our architecture to be classifier-independent, ensuring flexibility across different use cases. Therefore, the implementation allows the training task to be implemented as needed, allowing the operator to adjust it based on specific requirements. This is made possible by treating the training task as a standalone endpoint, independent of the classifier or underlying architecture. For instance, the operator could scale the training process without demanding significant modifications in our designed architecture. The newly generated model version is handled by a decoupled module responsible for the versioning and then made available to the deployed *Inference* modules for seamless integration and deployment. This periodic triggering of the model versioning allows the update of the SL model to be carried out under the MLOps framework. Our proposal has two main insights. First, we decouple inference, versioning, and updates, allowing incremental SL model updates without significantly impacting the performance of other modules. Second, we periodically perform model versioning and deployment, resulting in fewer model versions and increased inference throughput.

The modules that implement our proposal are described in the following subsections.

4.1. Model update and versioning

SL-oriented MLOps face significant challenges in handling frequent model updates and versioning, as these applications must operate in near real-time to accommodate continuously incoming data. Unlike traditional batch-oriented applications, which can store training data and perform model versioning at long intervals, SL models require incremental updates, generating a new model version with each evaluated event. This necessitates efficient serialization for versioning and deserialization to ensure updated models are readily available at the inference endpoints. Addressing these challenges is a challenging task to enable scalable and efficient SL-oriented applications within an MLOps framework.

In response, our proposed architecture introduces a triggering mechanism that manages the execution of model versioning and provisioning tasks (Fig. 2, *Periodically Triggered*). The trigger takes advantage of the characteristic of SL algorithms by examining their decision boundaries before triggering the model versioning task. For example, the Hoeffding Tree algorithm (Banar et al., 2023) will not split a tree leaf until certain events (the grace period parameter) have been observed. Moreover, even when model modifications are made on an event basis (e.g., Naive Bayes Yang, 2018), substantial model changes that affect inference are typically observed only after evaluating a significant number of samples.

Fig. 3(a) overviews our SL update mechanism. The process begins with an input SL model h and a predefined triggering frequency t. Initially, the number of evaluated events i is set to zero. The system then enters a continuous loop, ingesting events from the update queue. Each event x and its associated label y are used for incremental model updates. After each update, the system checks whether the number of processed events exceeds the predefined triggering frequency. A new model version is generated and serialized in the model repository if this condition is met.

Our scheme performs model versioning and deployment when a new model version is created. The versioning requires model serialization and storage, while the deployment sends the new model version to all deployed model inference endpoints in near real-time. This decoupling enables our scheme to provide high inference throughput even when using a low model trigger frequency, which may lead to the frequent generation of new model versions (Pipeline 3(a), t). Recalling that the model triggering frequency should be defined based on the operator's discretion, according to the used SL algorithm and required performance (latter evaluated in Section 6).

4.2. Model inference

MLOps-oriented inference must handle multiple users' requests at scale. However, in traditional SL scenarios, frequent model versioning and deployment can significantly impact inference throughput due to the computational costs of loading new model versions.

We implement the SL inference task to address this challenge as a distributed and decoupled service. In practice, inference is performed by multiple endpoints, each of which performs inference using its loaded SL model. Based on the load of user requests, this approach allows for horizontal scaling of inference tasks. A load balancer manages user requests, efficiently distributing them to the appropriate deployed inference endpoint to process the task (Fig. 2, *Inference Pipeline*).

When a new model version is available (see Section 4.1), each endpoint halts inference, requests the latest model version from the model versioning module, and resumes inference-related tasks. Given that our scheme relies on a triggering frequency mechanism for model versioning, the computational impact on inference is not significantly

degraded. This occurs because fewer SL model versions are generated over time, reducing the model deserialization efforts on the inference side.

Fig. 3(b) provides an overview of our SL update mechanism. The process begins with an input SL model h. The system then enters a continuous loop, where events are ingested for inference from the load balancer. Each ingested event x is processed by the current SL model h, which generates a predicted label y that is subsequently reported. After each inference, the system checks for the availability of a new model version in the model repository. If a new version is detected, it is retrieved and loaded through a deserialization procedure, and the inference task proceeds. This process runs continuously for each deployed inference endpoint.

4.3. Discussion

The implementation of the MLOps operation phase for SL-oriented applications is a challenging task. Our proposed model is built around two key principles. First, it decouples the inference, versioning, and update processes, enabling incremental SL model updates without compromising the performance of other system components. Second, it implements periodic model versioning and provisioning, which minimizes the number of model versions generated and enhances inference throughput. In practice, we leverage the behavior of SL algorithms, where typically their decision boundaries undergo significant changes only after evaluating a certain number of events. By aligning the versioning process with these natural adjustments, we reduce the frequency of updates, leading to fewer model versions without affecting the system's accuracy.

5. Prototype

A prototype was developed to validate the proposed architecture and assess the system's applicability in a production environment. As shown in Fig. 4, a controlled environment was created to replicate a real-world application. The prototype was implemented as a distributed Python application running as microservices encapsulated in containers, and orchestrated using Kubernetes (Bentaleb et al., 2021). The Kubernetes cluster also manages a container running Apache Kafka (Apache Software Foundation, 2011) v.3.7.0. This container receives messages with labeled events to be consumed by the update and versioning endpoint. The Microk8s tool (Canonical Ltd., 2018) v.1.24 was used to create and manage the Kubernetes cluster. The prototype comprises three main modules: *Inference Endpoints*, *Model Versioning*, and *Model Update*.

The *Inference Endpoints* are designed to be scalable on demand for executing the model inference task. We use a Kubernetes load balancer that appropriately distributes the load across each endpoint deployment. The endpoint is executed as a Docker container. It loads the selected SL model in River API v.0.21.2 (Montiel et al., 2021) format and ingests events to be evaluated using Flask API v.3.0 (Pallets Projects, 2010). It periodically checks for updated model versions. In this case, it downloads the updated model over the SMB protocol as it is provided and stored by the *Model Versioning* module.

The *Model Versioning* module stores and serves several SL model versions. It uses the MLFlow API v.2.16.0 (MLflow Project, 2018) for river SL model storage. It receives updated model versions generated by the *Model Update* module, stores them, and makes the updated model versions available to the *Inference Endpoint* via the SMB protocol.

Finally, the *Model Update* module is responsible for incremental SL model updates. We deploy a broker using the Kafka API v.3.7.0 (Apache Software Foundation, 2011), and continuously publish the events with their previously used labels for inference. Recall that we have two ingestion pipelines, one for the inference task, implemented as a web service, and the other for the update task, implemented as a publish–subscribe model. The module continuously receives updated events by listening to the corresponding Kafka topic. It uses these events to incrementally update the SL model. In addition, it periodically generates a new version of the model (Pipeline 3(a), t) for the *Model Version* module.

The proposed architecture addresses the SL challenges by enabling horizontal scaling of inference endpoints, seamless model loading without application restarts, incremental model updates, near-optimal model serialization, and efficient model version management. The system is built as containerized microservices orchestrated by Kubernetes. It is highly flexible and portable, suitable for both on-premises and cloud deployments. The inference engine efficiently loads model updates and scales horizontally, while the model update and versioning modules support incremental updates at a lower computational cost. They manage multiple model versions to ensure rapid model replacement across active endpoints. As a result, the architecture improves the system's ability to adapt to evolving data while maintaining stability and performance.

6. Evaluation

Our conducted experiments aim to answer the following Research Questions (RQs):

- ***RQ1:*** *What is the baseline accuracy of the SL model in an incremental update scenario?*
- ***RQ2:*** *What is the throughput of our proposed scheme for inference without incremental updates?*
- ***RQ3:*** *What is the impact of the frequency of model versioning on the accuracy and throughput?*
- ***RQ4:*** *How does endpoint parallelism affect throughput and accuracy?*

The next subsections describe the performance of the SL model building aspects.

6.1. The datasets and stream learning classifiers

We evaluated our proposal considering both synthetic and realistic datasets, as follows:

- **AGR-a** and **AGR-g** (Agrawal et al., 1993). A synthesized dataset based on the Agrawal generator. It comprises six nominal and three numerical features for a binary classification task. The dataset includes three abrupt concept drifts for the AGR-a dataset, and three gradual concept drifts for the AGR-g dataset;
- **YouChoose** (Asghar, 2016). A real-world dataset that captures user clicks and purchase events over several months was collected from an online retailer in 2014. It consists of 16 characteristics and a target attribute indicating whether a purchase was made;

We evaluated our scheme executed with 4 widely used SL classifiers, namely Naive Bayes (NB) (Yang, 2018), Adaptive Random Forest (ARF) (Gomes et al., 2017), Hoeffding Tree (HT) (Banar et al., 2023), and Adaptive Boosting (AdaBoost) (Oza, 2005). The Gaussian distribution technique was used for the NB. The HT was evaluated with a 200 grace period, information gain as a split criterion, and 100 maximum tree size. The ARF was evaluated with 10 base learners with the ADWIN drift detector, and information gain as the split criterion. The AdaBoost was assessed with 5 HT as the base learner with *gini* as the split criterion. The River API v.0.21.2 (see Fig. 4) was used to implement the selected classifiers. The parameters were set empirically, as often made in related works, and no significant differences were observed when varied.

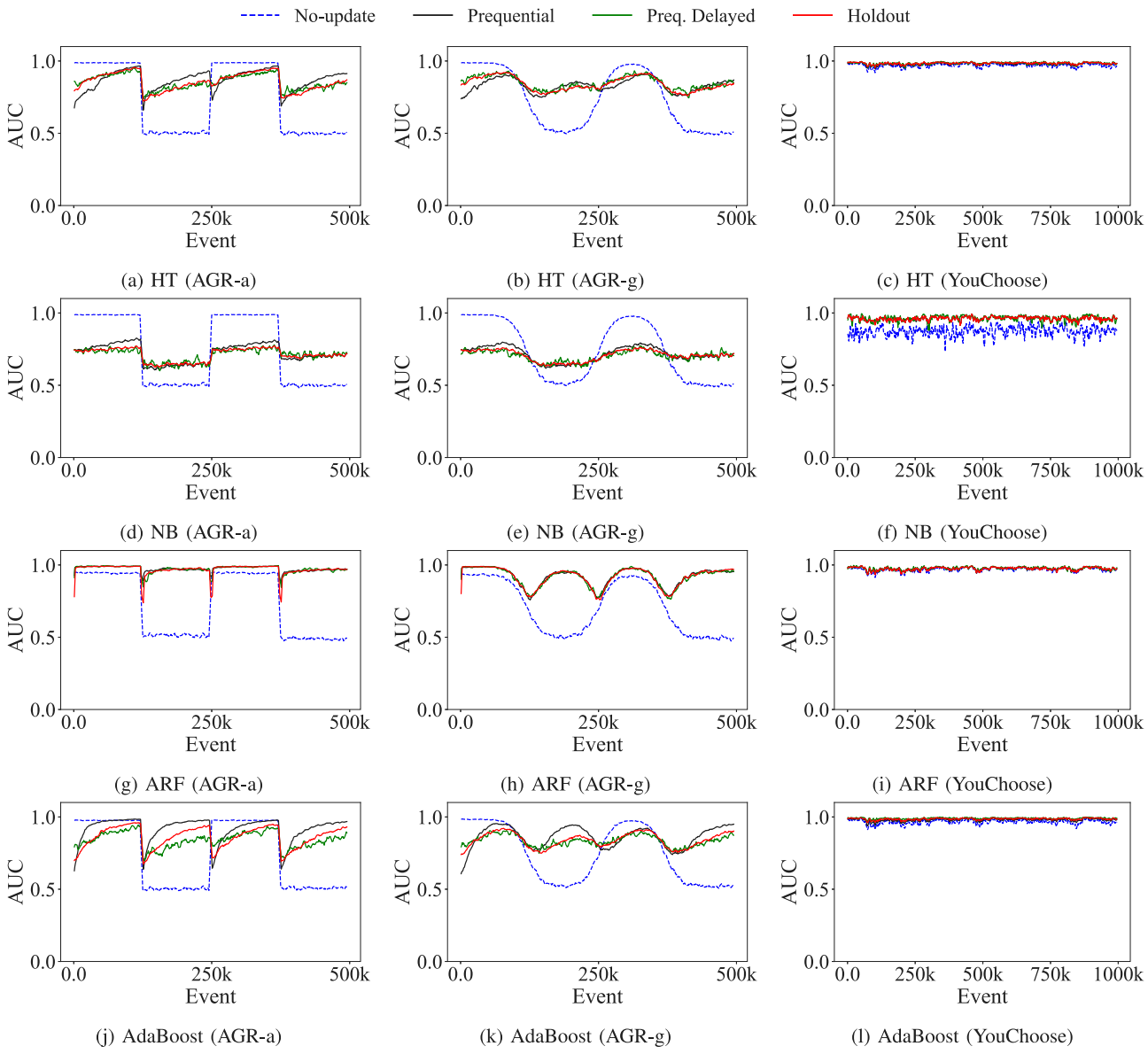


Fig. 5. Accuracy of selected SL classifiers on common datasets with and without incremental model updates, measured for every thousand-sample interval.

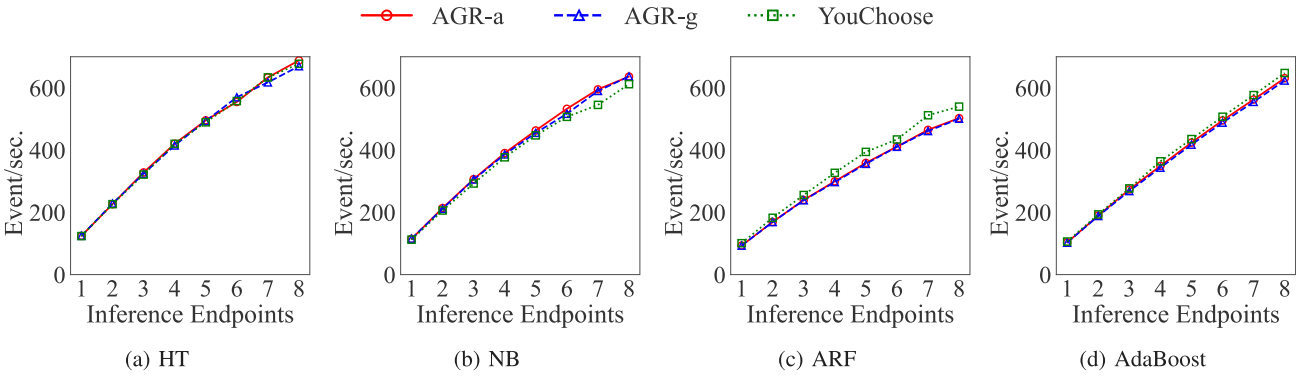


Fig. 6. Scalability of the *Inference Endpoint* of our proposed scheme without the application of incremental model updates, measured by the average number of events classified per second.

6.2. The stream challenge

Our first experiment aims at answering *RQ1*. It investigates the accuracy performance of the selected SL classifiers with an incremental update implementation. In practice, we establish a baseline detection accuracy before investigating the implementation aspects of our proposed MLOps architecture, as follows (see Fig. 1):

- *No-update*. The classifier is initially trained using the first 1% of the dataset. The resulting model then evaluates incoming samples without performing incremental updates;
- *Prequential*. Evaluate each incoming sample, apply it for incremental model updates, and proceed to process the next sample;
- *Holdout*. Process input in batches of 1000 samples. Each batch is partitioned such that 70% of the samples are allocated for testing, while the remaining 30% are utilized for incremental model updates;
- *Delayed*. Evaluate each incoming sample and apply it for incremental model updates with a delay of 1000 samples;

Therefore, we assess the accuracy of the selected classifiers using multiple evaluation setups, both with and without incremental model updates. According to the following equation, we assess the accuracy of the selected classifiers by their Area Under the Curve (AUC) values.

$$\mathrm{AUC} = \int_0^1 \mathrm{TPR}(t)\, d\mathrm{FPR}(t) \tag{1}$$

where the True-Positive Rate (TPR) denotes the ratio of positive events correctly classified, the False-Positive Rate (FPR) denotes the ratio of non-positive events incorrectly classified as positive, and t is the classification threshold set of the classifier. As commonly used in the literature, the AUC values are computed every thousand events. We used AUC because it provides a comprehensive measure of the model's ability to distinguish between positive and negative classes across all possible thresholds simultaneously. Since the classification operation point should be determined at the operator's discretion, AUC allows for an unbiased evaluation by considering both the TPR and FPR, regardless of class distribution.

Fig. 5 shows the obtained AUC values for the selected SL classifiers with and without incremental model updates being conducted. It is possible to note that the selected datasets require the execution of model updates for all evaluated SL classifiers, regardless of the utilized model update setup. For example, the HT on AGR-a (Fig. 5(a)) significantly degrades the measured AUC by an average of 0.13 throughout the evaluation period when no model updates are performed. Conversely, performing incremental model updates using the *prequential*, *holdout*, or *delayed* strategies can accommodate the evolving behavior of the selected datasets. Surprisingly, the selected classifiers achieve similar detection accuracies, irrespective of the underlying dataset and evaluation strategies. This demonstrates that the *delayed* update strategy can be effectively used for incremental model updates without significantly affecting model accuracy. As a result, this evaluation shows the impact that changes in behavior have on the classification performance of selected techniques.

6.3. Towards a MLOps for stream learning

Our second experiment aims to address *RQ2* by investigating the inference throughput of our proposed scheme without applying incremental updates. To this end, we scale the number of deployed *Inference Endpoints* on the deployed Kubernetes pods (see Fig. 4). We set the CPU limit to 0.5 for each docker in use. The experiment aims to investigate whether our implemented prototype can scale appropriately according to the number of deployed inference endpoints and the processing load. The goal is to ensure that our proposed architecture is effectively designed for distributed environments, demonstrating its ability to scale in response to adding new peers.

Fig. 6 shows the inference throughput according to the number of deployed endpoints. It can be observed that our proposed model has an improvement in inference throughput as the number of endpoints increases. For example, when the number of endpoints is increased from 1 to 8 on the AGR-a dataset, the inference throughput of the HT model increases from 124 to 688, an increase of 554% (Fig. 6(a)). Therefore, adding new peers results in an average increase of approximately 71% in inference throughput. This improvement is achieved with minimal variation across different datasets and classifiers. Thus, regardless of the SL application under consideration, our proposed model can effectively scale inference.

Our third experiment aims to answer *RQ3* and investigates how the frequency with which model versions are generated can affect both the accuracy of the inference and the throughput. In practice, we vary the frequency at which model versions are triggered (Pipeline 3(a), t). This impacts the frequency of generated new model versions, which can negatively affect inference throughput given the frequent need to deserialize new SL classifiers. We varied the model version triggering frequency (Pipeline 3(a), t), from 1000 to 2000 in intervals of 250 events. The test was executed with various active endpoints for each trigger frequency, from two to eight.

While the number of endpoints used in the test also influences the results, the primary focus of the evaluation is on the frequency of model availability. To achieve such a goal, we first examine the number of queued events on the update Kafka queue across different availability frequencies (Fig. 4, *Update Queue*). Recalling the proposed approach, unlike conventional methods found in the literature, the inference module can load the updated model into the endpoints' memory without recreating it, thus reducing the model update requirements on computational costs. The inference, update, and versioning endpoints also operate in parallel processes, allowing models to be loaded with lower computational overhead when replacing them. However, despite this computational advantage, deploying the model at very short intervals can still significantly impact the system, leading to a decline in predictive performance and system throughput

Fig. 7 shows the number of events on the update queue according to our scheme's used model version triggering frequency. An increase in the number of events in the update queue can be observed as the availability frequency parameter and the number of active endpoints in the system vary. However, when the number of endpoints reaches higher values, typically above 6, the queue often stabilizes or decreases for certain frequency parameter values. Further analysis of the results shows that for lightweight models like NB (Figs. 7(d), 7(e), and 7(f)) and HT (Figs. 7(a), 7(b), and 7(c)), higher availability frequency values (around 2000 events) have less impact on the update queue. Sometimes, the queue remains empty even with as many active endpoints. As the availability frequency decreases (≈1000 events), a consistent rise in the queue is observed, sometimes exceeding 250 thousand events waiting for consumption. In contrast, the ARF (Figs. 7(g), 7(h), and 7(i)) and AdaBoost (Figs. 7(j), 7(k), and 7(l)) classifiers does not exhibit such large fluctuations. This is because their inference task is computationally expensive, which slows event consumption during inference. As a result, fewer instances are sent to the update module, leading to smaller queue sizes.

Next, we examine the impact of model availability frequency on system throughput. To achieve this, we measured the number of events consumed per second and the number of available models for each case. Fig. 8 shows the event inference throughput, considering the processing of the entire pipeline across all availability frequency values and the number of active endpoints. Higher model version triggering frequency (≈2000 events) results in greater consumption of events per second compared to lower values. This occurs because their inference task is computationally expensive, slowing event consumption during inference. Consequently, a similar update queue size is observed regardless of the model versioning frequency, as the inference time is significantly higher than the time required for model serialization at the update

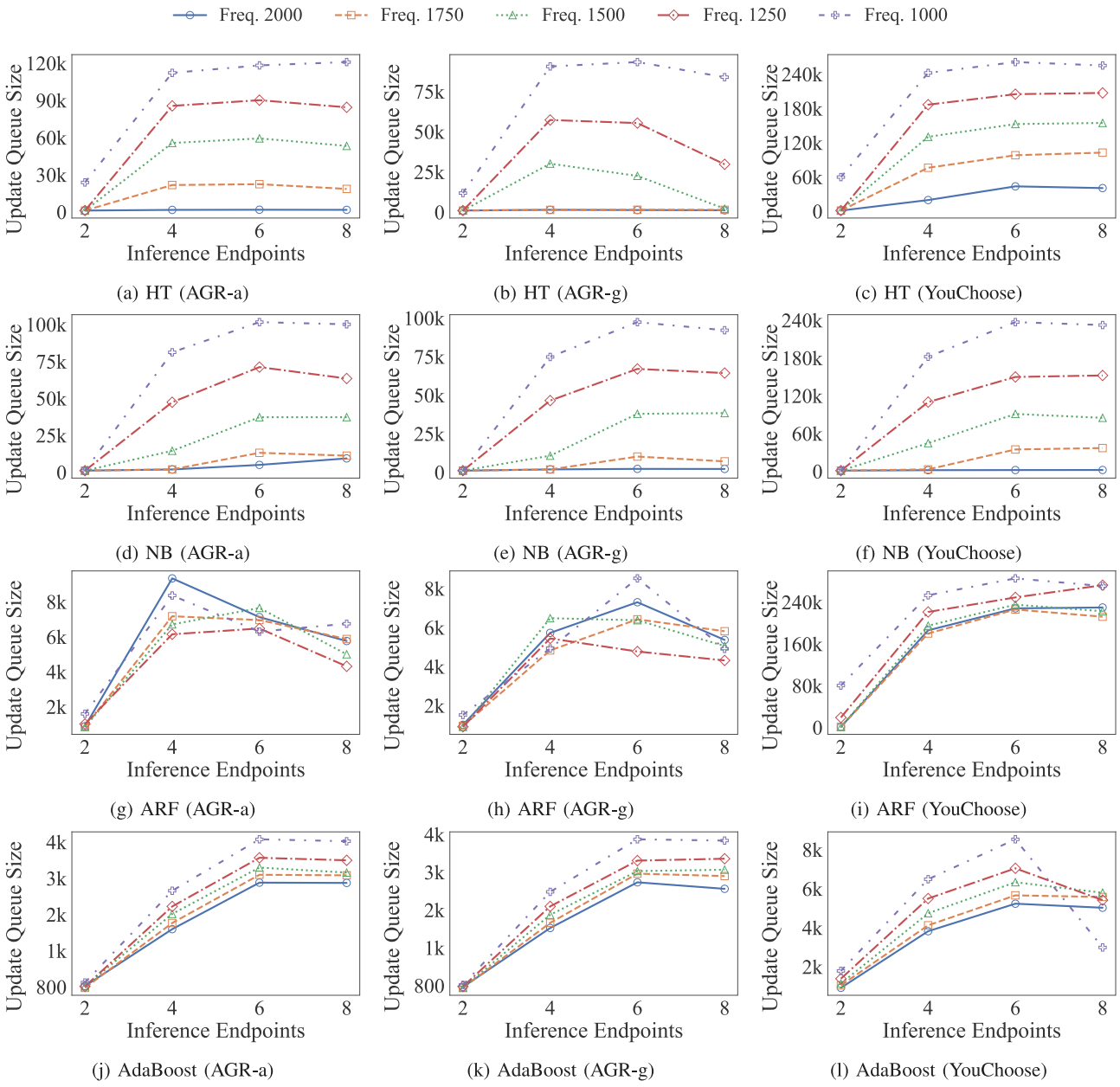


Fig. 7. Number of events in the update queue according to the model version trigger frequency (Pipeline 3(a), *t*).

endpoint and deserialization at the inference endpoints. As a result, fewer instances are sent to the update module, leading to smaller queue sizes.

Stabilization of the event consumption is observed when the number of active endpoints exceeds 6, similar to the results for the update queue. In practice, for the HT (Figs. 8(a), 8(b), and 8(c)) and NB (Figs. 8(d), 8(e), and 8(f)) learners, the inference throughput exceeds ≈650 instances per second when the model availability frequency is set to 2000 events and the number of active endpoints is 4 or more. As availability frequency decreases, event consumption gradually declines, with a maximum rate of about ≈480 events per second. In contrast, computational expensive models based on the ARF (Figs. 8(g), 8(h), and 8(i)), and AdaBoost (Figs. 8(j), 8(k), and 8(l)) exhibit lower event consumption rates due to their higher computation requirements. As a result, the model availability frequency significantly impacts system throughput. The system consumed nearly twice as many instances per second with the 2000 events parameter set than with the 1000 events configuration, especially with more than 4 active endpoints.

Fig. 9 shows the model version frequency according to the number of inference endpoints and the model version triggering. The system demonstrated high throughput, with model versioning occurring every ≈2.5, even when the availability frequency was set to higher values (2000 events) and the number of active endpoints was kept low (2 endpoints). Due to the significant system overhead in distributed environments, variations in the availability frequency parameter often had no direct impact on the number of models deployed per second. Models based on the ARF (Figs. 9(g), 9(h), and 9(i)), and AdaBoost (Figs. 9(j), 9(k), and 9(l)) learners were more affected by frequency changes due to their larger size, while lightweight models, such as those based on NB (Figs. 9(d), 9(e), and 9(f)) and HT (Figs. 9(a), 9(b), and 9(c)), maintained high deployment rates regardless of frequency adjustments.

Consequently, our proposed model can conduct model updates and versioning at a significantly high frequency. In practice, the achieved model versioning frequency can be relaxed according to the considered application dataset and model. For example, the ARF learner can adjust

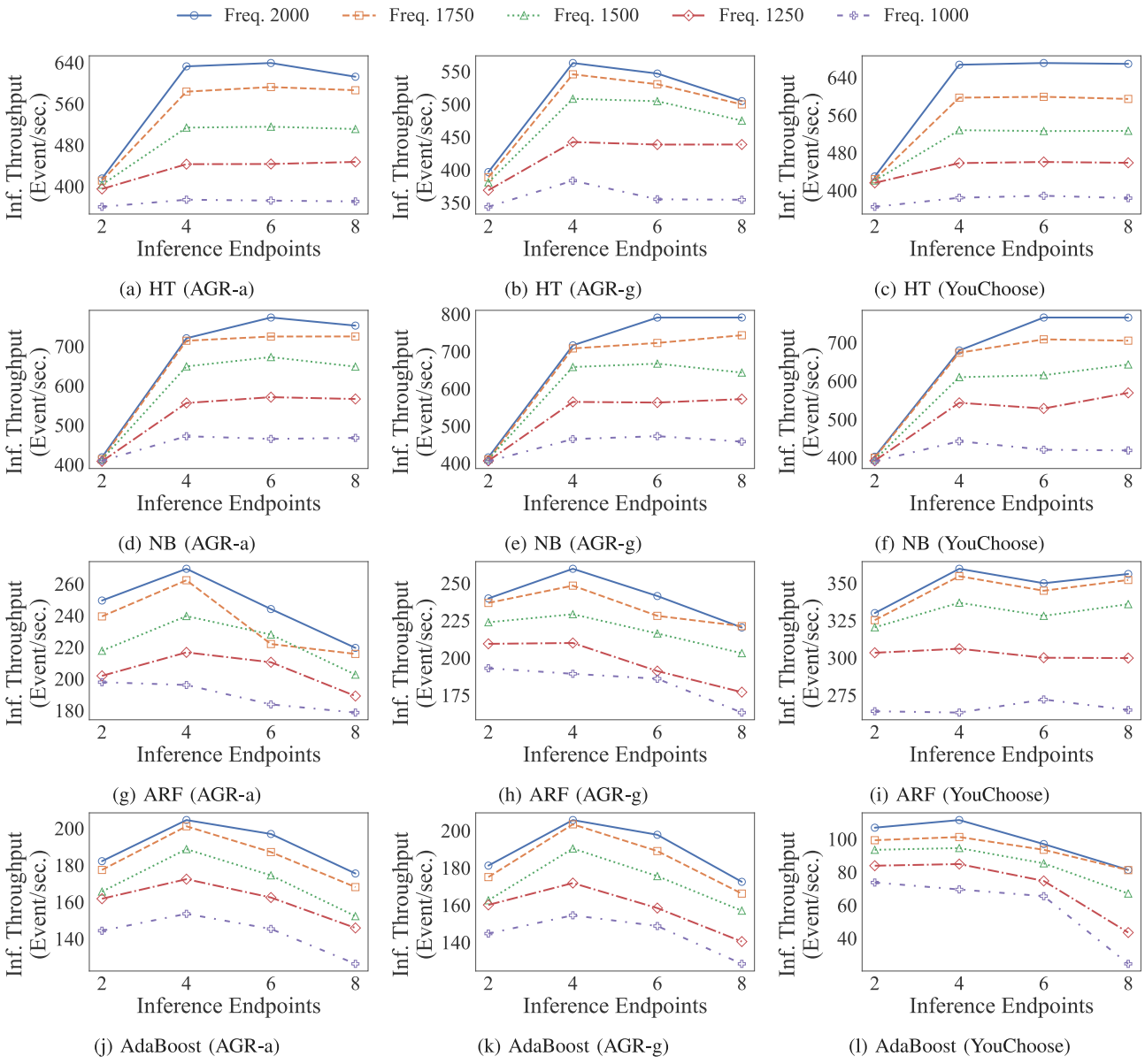


Fig. 8. Inference throughput according to the number of inference endpoints *vs*. the frequency of triggering the model version (Pipeline 3(a), *t*).

quicker to changes in the environment than other classifiers. As a result, model generation can be conducted less often, which results in a higher inference throughput without degrading the system's accuracy.

Finally, we answer *RQ4* by investigating how each chosen configuration can affect our scheme accuracy. Using the model version triggering frequency parameter, we investigated classifier accuracy as a function of the number of inference endpoints.

Fig. 10 shows the accuracy performance of the selected classifiers on the AGR-a dataset as a function of the number of inference endpoints used and the frequency of model versioning. The selected classifiers respond differently to updates due to the different characteristics of each learner. The NB (Figs. 10(e), 10(f), 10(g), and 10(h)), known for their high elasticity, retain old concepts and adapt minimally to new data. Even with lower availability frequency and maximum active endpoints, the AUC metric remains stable, indicating minimal variation and close alignment with the baseline. The ARF (Figs. 10(i), 10(j), 10(k), and 10(l)), and AdaBoost (Figs. 10(m), 10(n), 10(o), and 10(p)) learners, on the other hand, adapts quickly to current data and forgets older concepts. However, its large size slows inference throughput, which reduces the size of the update queue and allows updating with more recent data, which helps manage concept drift. Although occasional update queue spikes occur as drift detection mechanisms consume additional resources, system instance consumption normalizes as data concepts stabilize, keeping performance near baseline. Finally, changes in model availability frequency significantly affect the HT models.

Fig. 11 further investigates the accuracy distribution of the ARF SL classifier on the AGR-a dataset compared to the selected evaluation setups. The *Prequential* evaluation setup achieves the highest median accuracy, reaching an AUC of 0.9792. However, our proposed scheme demonstrates a similar accuracy distribution to the *Prequential* setup, even when employing a high model versioning frequency. For instance, with a model versioning frequency of one thousand events and eight inference endpoints (Fig. 10(l)), our model achieves a median AUC of 0.9754, representing only a 0.39% decrease compared to the best-performing evaluation setup. Additionally, the accuracy distribution range remains comparable to the *Prequential* setup, with an interquartile range of 0.0259 versus 0.0249, an increase of merely 4%. It is important to note that these results are obtained while performing near real-time model updates and versioning, along with inference at scale.

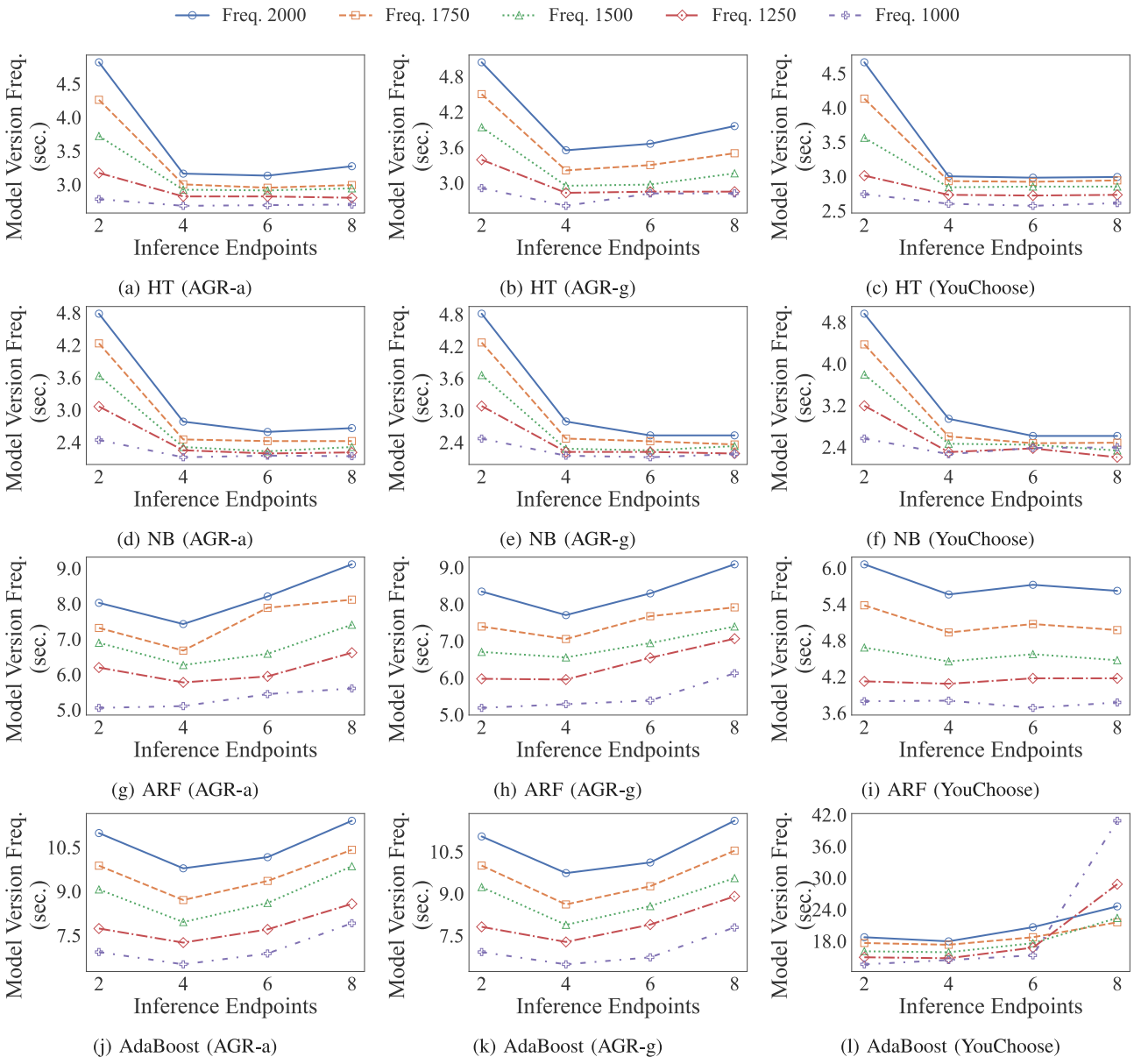


Fig. 9. New model version frequency provision, according to the number of inference endpoints *vs*. the model version trigger frequency (Pipeline 3(a), *t*).

6.4. Discussion

The conducted experiments showed that our proposed scheme enabled the implementation of SL algorithms under the MLOps framework. In practice, the following remarks were observed.

- ***RQ1***. Incremental model updates, in which the model is updated and then used to predict the next event, allow for rapid adaptation to changes in the data while maintaining satisfactory predictive performance throughout the data flow (Fig. 5). This incremental update behavior is often a must to ensure reliable SL performance;
- ***RQ2***. Scaling the number of inference endpoints significantly increases the inference throughput. However, this increase is not linear due to the distributed system overhead (Fig. 6). Actual performance deviates from the ideal as the number of active endpoints increases, with actual performance averaging 30% less than the ideal for 8 active endpoints. Due to their size and complexity, models based on the ARF learner perform less than their counterparts. The scalability of the inference throughput does not significantly fluctuate according to the used dataset, attesting to the general purpose of our architecture;
- ***RQ3***. Model versioning frequency has an impact on the number of events in the update queue, which significantly degrades other results (Fig. 7). Although system overhead can affect these values, higher model versioning frequencies tend to increase event consumption and the number of updated models per second. Model frequency also has different effects on the predictive performance of models, with each type of learner responding differently to changes in frequency and update queue size. In general, higher values for the model frequency trigger will lead to a better predictive performance than lower values.
- ***RQ4***. Increasing the number of active endpoints directly affects the update queue, inference throughput, and model predictive performance. As the number of endpoints increases, there is a tendency for the update queue to grow significantly. This expansion leads to decreased system throughput and a deterioration in predictive performance. The growing queue can result in outdated models and slower recovery as the time between instance arrival and consumption increases.

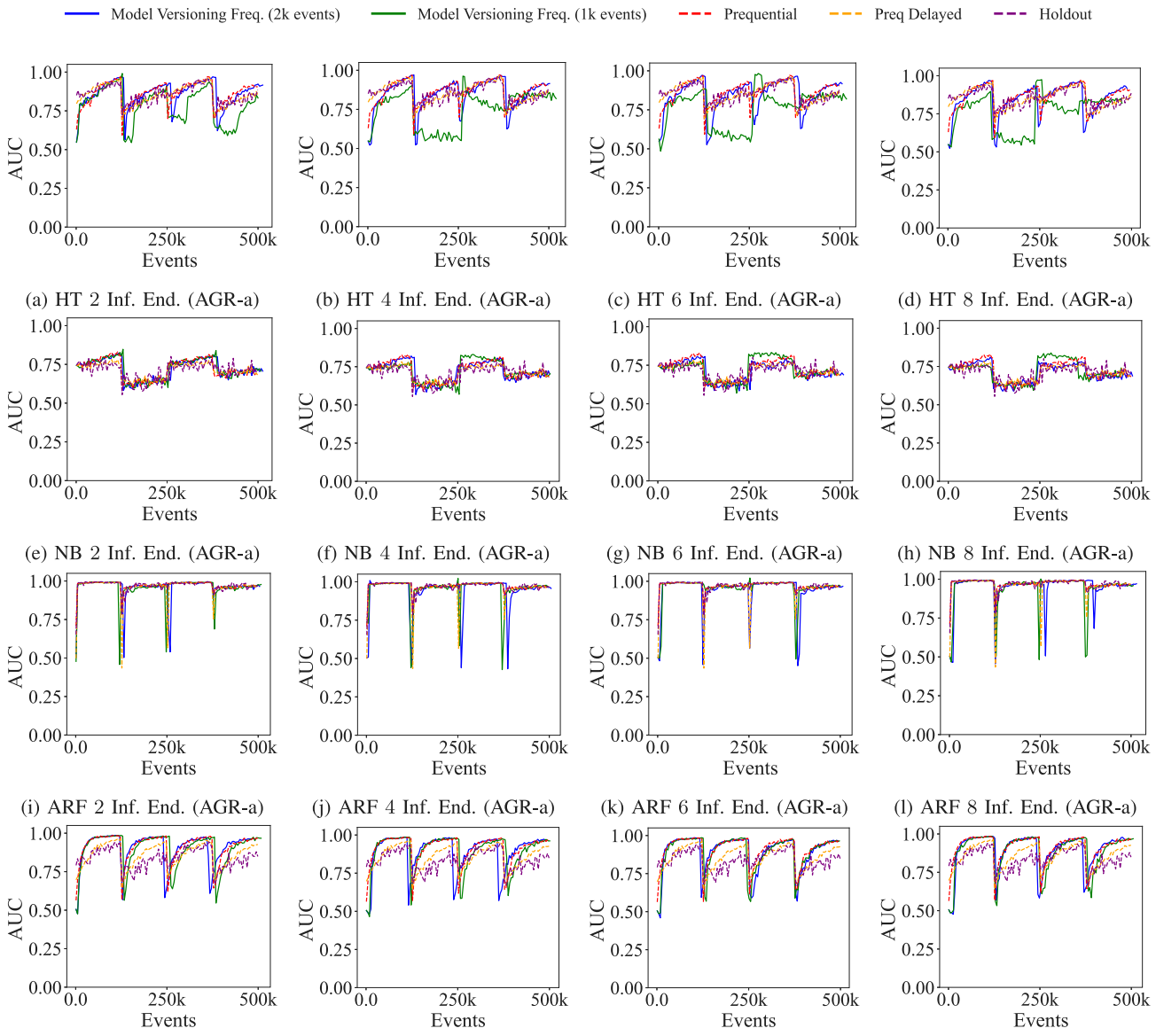


Fig. 10. AUC values for each selected classifier on the AGR-a dataset as a function of the number of inference endpoints and the model version trigger threshold. Our proposed scheme achieves similar baseline accuracy while implementing distributed MLOps.

The analysis of the results indicates that careful configuration of parameters, such as availability frequency and the number of endpoints, is needed for balancing system throughput, queue size, and model predictive performance. Proper adjustment of these parameters helps to optimize event consumption per second, minimize processing delays, and prevent excessive data accumulation while maintaining model accuracy. Additionally, each learner's intrinsic characteristics and model size significantly affect system behavior. For instance, NB models, which are less sensitive to parameter variations, show more stable performance. Computationally heavier models, such as those based on ARF and AdaBoost, exhibit less performance fluctuation due to parameter changes. In contrast, HT models, with their favorable balance of elasticity and relatively small sizes, are more impacted by parameter variations. It is essential to note that these results were obtained under significant hardware constraints to highlight the main system bottlenecks. Overall, the choice of configuration parameters is fundamental to the system's performance, with optimal results achieved through detailed and customized parameter settings that account for learner characteristics and production environment needs.

The frequency of model versioning is an important aspect of balancing system performance and accuracy. Operators should define a model versioning triggering frequency that ensures accuracy is maintained in proportion to the number of deployed inference endpoints. However, it is essential to consider the trade-offs of frequent model versioning. While more frequent updates may improve responsiveness to changes in data, they can also negatively affect inference throughput by introducing delays and increasing the load on the model update queue. As a result, the operator should consider finding the operation point that can optimize both accuracy and inference throughput when deploying our proposed architecture.

7. Conclusion

This paper presents a novel architecture for deploying SL models under the MLOps framework, which incorporates incremental updates and frequent model versioning. The architecture suits various systems, including cloud environments, because it is based on containerized microservices that ensure portability and flexibility. Unlike traditional

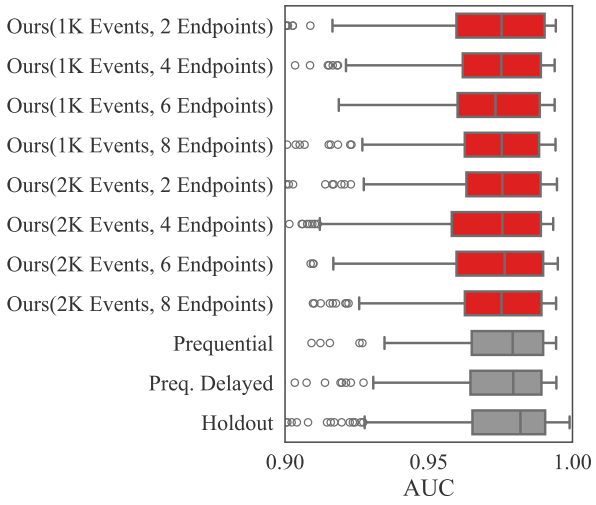


Fig. 11. Accuracy distribution of the ARF classifier on the AGR-a dataset considering multiple architecture configurations *vs* traditional evaluation setups.

approaches that periodically update models in batches, our architecture addresses the challenge of continuously adapting to data changes through incremental updates. The implementation provides a scalable inference service that maintains efficiency while addressing key issues such as rapidly updating models, managing multiple model versions, and handling large data streams. Extensive experiments validated the architecture, highlighting the critical role of parameter configuration in the balance between throughput, queue size, and predictive performance. The architecture demonstrated robustness, maintaining low model availability times and high predictive accuracy, even in demanding scenarios

Future work will evaluate the architecture's performance with multiple concurrent models, examine its ability to manage different data flows and model versioning and optimize its adaptability and scalability for more complex situations.

The source code and used datasets are publicly available for download at https://github.com/mgrodrigues001/MLOps-Architecture.

CRediT authorship contribution statement

Miguel G. Rodrigues: Writing – original draft, Methodology, Investigation, Data curation. **Eduardo K. Viegas:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Conceptualization. **Altair O. Santin:** Writing – review & editing, Supervision. **Fabricio Enembreck:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially sponsored by the Brazilian National Council for Scientific and Technological Development (CNPq), grants n° 304990/2021-3, 407879/2023-4, 442262/2024-8, and 302937/2023-4.

Data availability

Data will be made available on request.

References

Agrahari, S., Singh, A.K., 2022. Concept drift detection in data stream mining: A literature review. J. King Saud Univ. - Comput. Inf. Sci. 34 (10), 9523–9540, [Online]. Available: http://dx.doi.org/10.1016/j.jksuci.2021.11.006.

Agrawal, R., Imielinski, T., Swami, A., 1993. Database mining: a performance perspective. IEEE Trans. Knowl. Data Eng. 5 (6), 914–925, [Online]. Available: http://dx.doi.org/10.1109/69.250074.

Amazon Web Services, I., 2017. Amazon SageMaker. [Online]. Available: https://aws.amazon.com/sagemaker/.

Anandan, S., Bogoevici, M., Renfro, G., Gopinathan, I., Peralta, P., 2015. Spring XD: a modular distributed stream and batch processing system. In: Proceedings of the 9th ACM International Conference on Distributed Event-Based Systems. DEBS '15, ACM, pp. 217–225, [Online]. Available: http://dx.doi.org/10.1145/2675743.2771879.

Apache Software Foundation, 2006a. Apache hadoop. [Online]. Available: https://hadoop.apache.org/.

Apache Software Foundation, 2006b. Hadoop distributed file system (HDFS). [Online]. Available: https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html.

Apache Software Foundation, 2010. Apache zookeeper. [Online]. Available: https://zookeeper.apache.org/.

Apache Software Foundation, 2011. Apache kafka. [Online]. Available: https://kafka.apache.org/.

Apache Software Foundation, 2013. Apache spark. [Online]. Available: https://spark.apache.org/.

Apache Software Foundation, 2014. Storm. [Online]. Available: https://storm.apache.org/.

Apache Software Foundation, 2014a. Apache flink: Stateful computations over data streams. [Online]. Available: https://flink.apache.org/.

Apache Software Foundation, 2014b. MLlib: Machine learning library. [Online]. Available: https://spark.apache.org/mllib/.

Apache Software Foundation, 2014c. SAMOA - scalable advanced massive online analysis. [Online]. Available: https://incubator.apache.org/projects/samoa.html.

Apache Software Foundation, 2018. Samza. [Online]. Available: https://samza.apache.org/.

Asghar, N., 2016. Yelp dataset challenge: Review rating prediction. [Online]. Available: https://arxiv.org/abs/1605.05362.

Ashmore, R., Calinescu, R., Paterson, C., 2021. Assuring the machine learning lifecycle: Desiderata, methods, and challenges. ACM Comput. Surv. 54 (5), 1–39, [Online]. Available: http://dx.doi.org/10.1145/3453444.

Banar, F., Tabatabaei, A., Saleh, M., 2023. Stream data classification with hoeffding tree: An ensemble learning approach. In: 2023 9th International Conference on Web Research. ICWR, IEEE, [Online]. Available: http://dx.doi.org/10.1109/ICWR57742.2023.10139228.

Barddal, J.P., Gomes, H.M., Enembreck, F., Pfahringer, B., 2017. A survey on feature drift adaptation: Definition, benchmark, challenges and future directions. J. Syst. Softw. 127, 278–294, [Online]. Available: http://dx.doi.org/10.1016/j.jss.2016.07.005.

Barry, M., Montiel, J., Bifet, A., Wadkar, S., Manchev, N., Halford, M., Chiky, R., Jaouhari, S.E., Shakman, K.B., Fehaily, J.A., Le Deit, F., Tran, V.-T., Guerizec, E., 2023. StreamMLOps: Operationalizing online learning for big data streaming & real-time applications. In: 2023 IEEE 39th International Conference on Data Engineering. ICDE, IEEE, [Online]. Available: http://dx.doi.org/10.1109/ICDE55515.2023.00272.

Baylor, D., Breck, E., Cheng, H.-T., Fiedel, N., Foo, C.Y., Haque, Z., Haykal, S., Ispir, M., Jain, V., Koc, L., Koo, C.Y., Lew, L., Mewald, C., Modi, A.N., Polyzotis, N., Ramesh, S., Roy, S., Whang, S.E., Wicke, M., Wilkiewicz, J., Zhang, X., Zinkevich, M., 2017. TFX: A TensorFlow-based production-scale machine learning platform. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '17, ACM, [Online]. Available: http://dx.doi.org/10.1145/3097983.3098021.

Bentaleb, O., Belloum, A.S.Z., Sebaa, A., El-Maouhab, A., 2021. Containerization technologies: taxonomies, applications and challenges. J. Supercomput. 78 (1), 1144–1181, [Online]. Available: http://dx.doi.org/10.1007/s11227-021-03914-1.

Bifet, A., Maniu, S., Qian, J., Tian, G., He, C., Fan, W., 2015. StreamDM: Advanced data mining in spark streaming. In: 2015 IEEE International Conference on Data Mining Workshop. ICDMW, IEEE, [Online]. Available: http://dx.doi.org/10.1109/ICDMW.2015.140.

Burgueño-Romero, A.M., Barba-González, C., Aldana-Montes, J.F., 2025. Big data-driven MLOps workflow for annual high-resolution land cover classification models. Future Gener. Comput. Syst. 163, 107499, [Online]. Available: http://dx.doi.org/10.1016/j.future.2024.107499.

Canonical Ltd., 2018. Microk8s: Lightweight kubernetes for developers and DevOps. [Online]. Available: https://microk8s.io/.

Carcillo, F., Dal Pozzolo, A., Le Borgne, Y.-A., Caelen, O., Mazzer, Y., Bontempi, G., 2018. SCARFF: A scalable framework for streaming credit card fraud detection with spark. Inf. Fusion 41, 182–194, [Online]. Available: http://dx.doi.org/10.1016/j.inffus.2017.09.005.

Chaoji, V., Rastogi, R., Roy, G., 2016. Machine learning in the real world. Proc. VLDB Endow. 9 (13), 1597–1600, [Online]. Available: http://dx.doi.org/10.14778/3007263.3007318.

Crankshaw, D., Wang, X., Zhou, G., Franklin, M.J., Gonzalez, J.E., Stoica, I., 2017. Clipper: a low-latency online prediction serving system. In: Proceedings of the 14th USENIX Conference on Networked Systems Design and Implementation. NSDI '17, USENIX Association, USA, pp. 613–627.

Elastic, 2010. Elasticsearch. [Online]. Available: https://www.elastic.co/elasticsearch/.

Elastic, 2013. Kibana. [Online]. Available: https://www.elastic.co/kibana/.

Fayos-Jordan, R., Felici-Castell, S., Segura-Garcia, J., Lopez-Ballester, J., Cobos, M., 2020. Performance comparison of container orchestration platforms with low cost devices in the fog, assisting internet of things applications. J. Netw. Comput. Appl. 169, 102788, [Online]. Available: http://dx.doi.org/10.1016/j.jnca.2020.102788.

Gama, J., Sebastião, R., Rodrigues, P.P., 2012. On evaluating stream learning algorithms. Mach. Learn. 90 (3), 317–346, [Online]. Available: http://dx.doi.org/10.1007/s10994-012-5320-9.

Garcia-Rodriguez, S., Alshaer, M., Gouy-Pailler, C., 2020. STREAMER: A powerful framework for continuous learning in data streams. In: Proceedings of the 29th ACM International Conference on Information & Knowledge Management. CIKM '20, ACM, [Online]. Available: http://dx.doi.org/10.1145/3340531.3417427.

Garg, S., Pundir, P., Rathee, G., Gupta, P., Garg, S., Ahlawat, S., 2021. On continuous integration / continuous delivery for automated deployment of machine learning models using MLOps. In: 2021 IEEE Fourth International Conference on Artificial Intelligence and Knowledge Engineering. AIKE, IEEE, [Online]. Available: http://dx.doi.org/10.1109/AIKE52691.2021.00010.

Gomes, H.M., Barddal, J.P., Enembreck, F., Bifet, A., 2017. A survey on ensemble learning for data stream classification. ACM Comput. Surv. 50 (2), 1–36, [Online]. Available: http://dx.doi.org/10.1145/3054925.

Gomes, H.M., Read, J., Bifet, A., Barddal, J.P., Gama, J., 2019. Machine learning for streaming data: state of the art, challenges, and opportunities. ACM SIGKDD Explor. Newsl. 21 (2), 6–22, [Online]. Available: http://dx.doi.org/10.1145/3373464.3373470.

Gonçalves, P.M., de Carvalho Santos, S.G., Barros, R.S., Vieira, D.C., 2014. A comparative study on concept drift detectors. Expert Syst. Appl. 41 (18), 8144–8156, [Online]. Available: http://dx.doi.org/10.1016/j.eswa.2014.07.019.

Google, 2018. Google cloud AI plataform. [Online]. Available: https://console.cloud.google.com/marketplace/product/google-cloud-platform/cloud-machine-learning-engine?project=curso-403115.

Google Brain Team, 2015. TensorFlow: An open-source machine learning framework. [Online]. Available: https://www.tensorflow.org/.

Horchulhack, P., Viegas, E.K., Santin, A.O., Ramos, F.V., Tedeschi, P., 2024. Detection of quality of service degradation on multi-tenant containerized services. J. Netw. Comput. Appl. 224, 103839, [Online]. Available: http://dx.doi.org/10.1016/j.jnca.2024.103839.

InfluxData, 2013. Influxdb. [Online]. Available: https://www.influxdata.com/products/influxdb/.

Jamil, W., Duong, N.-C., Wang, W., Mansouri, C., Mohamad, S., Bouchachia, A., 2018. Scalable online learning for flink: SOLMA library. In: Proceedings of the 12th European Conference on Software Architecture: Companion Proceedings. ECSA '18, Vol. 58, ACM, pp. 1–4, [Online]. Available: http://dx.doi.org/10.1145/3241403.3241438.

Kiaee, F., Arianyan, E., 2025. Joint VM and container consolidation with auto-encoder based contribution extraction of decision criteria in edge-cloud environment. J. Netw. Comput. Appl. 233, 104049, [Online]. Available: http://dx.doi.org/10.1016/j.jnca.2024.104049.

Komisarek, M., Choraś, M., Kozik, R., Pawlicki, M., 2020. Real-time stream processing tool for detecting suspicious network patterns using machine learning. In: Proceedings of the 15th International Conference on Availability, Reliability and Security. In: ARES 2020, ACM, [Online]. Available: http://dx.doi.org/10.1145/3407023.3409189.

Kranjc, J., Orač, R., Podpečan, V., Lavrač, N., Robnik-Šikonja, M., 2017. ClowdFlows: Online workflows for distributed big data mining. Future Gener. Comput. Syst. 68, 38–58, [Online]. Available: http://dx.doi.org/10.1016/j.future.2016.07.018.

Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G., 2018. Learning under concept drift: A review. IEEE Trans. Knowl. Data Eng. 1, [Online]. Available: http://dx.doi.org/10.1109/TKDE.2018.2876857.

Markov, I.L., Wang, H., Kasturi, N.S., Singh, S., Garrard, M.R., Huang, Y., Yuen, S.W.C., Tran, S., Wang, Z., Glotov, I., Gupta, T., Chen, P., Huang, B., Xie, X., Belkin, M., Uryasev, S., Howie, S., Bakshy, E., Zhou, N., 2022. Looper: An end-to-end ML platform for product decisions. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. KDD '22, Vol. 28, ACM, pp. 3513–3523, [Online]. Available: http://dx.doi.org/10.1145/3534678.3539059.

Martín, C., Langendoerfer, P., Zarrin, P.S., Díaz, M., Rubio, B., 2022. Kafka-ML: Connecting the data stream with ML/AI frameworks. Future Gener. Comput. Syst. 126, 15–33, [Online]. Available: http://dx.doi.org/10.1016/j.future.2021.07.037.

Microsoft Corporation, 2018. Azure machine learning. [Online]. Available: https://azure.microsoft.com/.

Minon, R., Diaz-de Arcaya, J., Torre-Bastida, A.I., Zarate, G., Moreno-Fernandez-de Leceta, A., 2022. Mlpacker: A unified software tool for packaging and deploying atomic and distributed analytic pipelines. In: 2022 7th International Conference on Smart and Sustainable Technologies. SpliTech, Vol. 20, IEEE, pp. 1–6, [Online]. Available: http://dx.doi.org/10.23919/SpliTech55088.2022.9854211.

MLflow Project, 2018. An open source platform for the machine learning lifecycle. [Online]. Available: https://mlflow.org/.

Montiel, J., Halford, M., Mastelini, S.M., Bolmier, G., Sourty, R., Vaysse, R., Zouitine, A., Gomes, H.M., Read, J., Abdessalem, T., et al., 2021. River: machine learning for streaming data in python. [Online]. Available: https://riverml.xyz/.

Montiel, J., Read, J., Bifet, A., Abdessalem, T., 2018. Scikit-multiflow: A multi-output streaming framework. J. Mach. Learn. Res. 19 (72), 1–5, [Online]. Available: http://jmlr.org/papers/v19/18-251.html.

Moya, A.R., Veloso, B., Gama, J., Ventura, S., 2023. Improving hyper-parameter self-tuning for data streams by adapting an evolutionary approach. Data Min. Knowl. Discov. 38 (3), 1289–1315, [Online]. Available: http://dx.doi.org/10.1007/s10618-023-00997-7.

Neto, E.C.P., Dadkhah, S., Sadeghi, S., Molyneaux, H., Ghorbani, A.A., 2024. A review of machine learning (ML)-based IoT security in healthcare: A dataset perspective. Comput. Commun. 213, 61–77, [Online]. Available: http://dx.doi.org/10.1016/j.comcom.2023.11.002.

Oza, N., Online Bagging and Boosting. In: 2005 IEEE International Conference on Systems, Man and Cybernetics. Vol. 3, IEEE, pp. 2340–2345, [Online]. Available: http://dx.doi.org/10.1109/ICSMC.2005.1571498.

Paleyes, A., Urma, R.-G., Lawrence, N.D., 2022. Challenges in deploying machine learning: A survey of case studies. ACM Comput. Surv. 55 (6), 1–29, [Online]. Available: http://dx.doi.org/10.1145/3533378.

Pallets Projects, 2010. Flask: A python microframework. [Online]. Available: https://flask.palletsprojects.com/.

Pareek, A., Zhang, B., Khaladkar, B., 2019. A demonstration of striim a streaming integration and intelligence platform. In: Proceedings of the 13th ACM International Conference on Distributed and Event-Based Systems. DEBS '19, ACM, pp. 236–239, [Online]. Available: http://dx.doi.org/10.1145/3328905.3332519.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E., 2011. Scikit-learn: Machine learning in python. J. Mach. Learn. Res. 12, 2825–2830.

Pivotal Software, 2007. RabbitMQ. [Online]. Available: https://www.rabbitmq.com/.

Pruneski, J.A., Williams, R.J., Nwachukwu, B.U., Ramkumar, P.N., Kiapour, A.M., Martin, R.K., Karlsson, J., Pareek, A., 2022. The development and deployment of machine learning models. Knee Surg. Sport. Traumatol. Arthrosc. 30 (12), 3917–3923, [Online]. Available: http://dx.doi.org/10.1007/s00167-022-07155-4.

Python Software Foundation, 2020. Creme. [Online]. Available: https://pypi.org/project/creme/.

PyTorch Foundation, 2016. Pytorch: An open-source machine learning library. [Online]. Available: https://pytorch.org/.

Ramanath, R., Salomatin, K., Gee, J.D., Talanine, K., Dalal, O., Polatkan, G., Smoot, S., Kumar, D., 2021. Lambda learner: Fast incremental learning on data streams. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. KDD '21, ACM, [Online]. Available: http://dx.doi.org/10.1145/3447548.3467172.

Redis Labs, 2009. Redis. [Online]. Available: https://redis.io/.

Seldon Technologies Limited, 2014. Seldon. [Online]. Available: https://www.seldon.io/.

Shinde, P.P., Shah, S., 2018. A review of machine learning and deep learning applications. In: 2018 Fourth International Conference on Computing Communication Control and Automation. ICCUBEA, IEEE, [Online]. Available: http://dx.doi.org/10.1109/ICCUBEA.2018.8697857.

Suárez-Cetrulo, A.L., Quintana, D., Cervantes, A., 2023. A survey on machine learning for recurring concept drifting data streams. Expert Syst. Appl. 213, 118934, [Online]. Available: http://dx.doi.org/10.1016/j.eswa.2022.118934.

The Apache Software Foundation, 2014. Apache airflow. [Online]. Available: https://airflow.apache.org/.

The Kubeflow Authors, 2018. Kubeflow. [Online]. Available: https://www.kubeflow.org/.

Upadhyaya, S.R., 2013. Parallel approaches to machine learning—A comprehensive survey. J. Parallel Distrib. Comput. 73 (3), 284–292, [Online]. Available: http://dx.doi.org/10.1016/j.jpdc.2012.11.001.

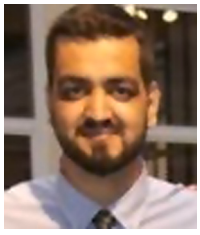
The University of Waikato, 2010. MOA - massive online analysis. [Online]. Available: https://moa.cms.waikato.ac.nz/.

Xu, D., Wu, D., Xu, X., Zhu, L., Bass, L., 2015. Making real time data analytics available as a service. In: Proceedings of the 11th International ACM SIGSOFT Conference on Quality of Software Architectures. CompArch '15, ACM, pp. 73–82, [Online]. Available: http://dx.doi.org/10.1145/2737182.2737186.

Yadwadkar, N.J., Romero, F., Li, Q., Kozyrakis, C., 2019. A case for managed and model-less inference serving. In: Proceedings of the Workshop on Hot Topics in Operating Systems. HotOS '19, ACM, [Online]. Available: http://dx.doi.org/10.1145/3317550.3321443.

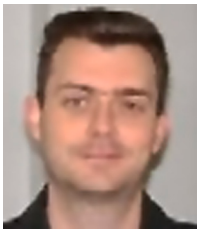
Yang, F.-J., 2018. An implementation of naive Bayes classifier. In: 2018 International Conference on Computational Science and Computational Intelligence. CSCI, IEEE, [Online]. Available: http://dx.doi.org/10.1109/CSCI46756.2018.00065.

Miguel G. Rodrigues is a Computer Science MSc. candidate at the Pontifical Catholic University of Paraná (PUCPR). He received his bachelor's degree in Big Data and Data Science from PUCPR in 2021. His research includes stream learning applications, Big Data, Distributed Systems, and MLOps.



Eduardo K. Viegas received his BS in computer science in 2013, the MSC in computer science in 2016 from PUCPR, and the Ph.D. degree from PUCPR in 2018. He is an associate professor of the Graduate Program in Computer Science (PPGIa). His research interests include machine learning, network analytics, and computer security.

Altair O. Santin received the BS degree in Computer Engineering from the PUCPR in 1992, the M.Sc. degree from UTFPR in 1996, and the Ph.D. degree from UFSC in 2004. He is a full professor of the Graduate Program in Computer Science (PPGIa) and head of the Security & Privacy Lab (SecPLab) at PUCPR. He is an IEEE, ACM, and Brazilian Computer Society member.



Fabricio Enembreck holds a degree in Computer Science from the State University of Ponta Grossa (1997), a master's degree in Computer Science from the Pontifical Catholic University of Paraná (1999), and a Ph.D. in Information and Systems Technologies from the Universite de Technologie de Compiegne (2003). He is currently an Full Professor at the Pontifical Catholic University of Paraná. He has experience in the area of Computer Science, with an emphasis on Computer Systems. He works mainly on the following topics: assistant agents, autonomous agents, adaptive autonomous agents, distributed artificial intelligence, multi-agent systems, and machine learning.